

24CSA212 – Full Stack Frameworks

INTRODUCTION TO REACT JS

Introduction to React JS

- React is a front-end JavaScript library.
- React was developed by the Facebook Software Engineer Jordan Walke.
- React is also known as React.js or ReactJS.
- React is a tool for building UI components.

How does React Work?

- React creates a VIRTUAL DOM in memory.
- Instead of manipulating the browser's DOM directly, React creates a virtual DOM in memory, where it does all the necessary manipulating, before making the changes in the browser DOM.
- React only changes what needs to be changed!
- React finds out what changes have been made, and changes only what needs to be changed.

History & Evolution of React

SECTION 01 — INTRODUCTION

2011

React created internally at Facebook by Jordan Walke

2013

React open-sourced at JSConf US — community skeptical

2015

React Native released; React v0.13 introduces ES6 classes

2016

React v15 — improved DOM rendering performance

2017

React v16 (Fiber) — complete architecture rewrite

2018

React Hooks introduced in v16.8 — game changer!

2020

React v17 — no new features, improved upgrade path

2022

React v18 — Concurrent Mode, Suspense, useTransition

2024

React 19 — Server Components, Actions, new hooks

Key Features

- **Component-Based Architecture**

- UI is broken into small, reusable pieces called **components**.
- Each component manages its own state and logic.

- **Virtual DOM**

- React uses a **virtual representation of the DOM** to efficiently update only the parts of the UI that change, improving performance.

- **Declarative UI**

- You describe **what** the UI should look like, and React updates it automatically when data changes.

- **Unidirectional Data Flow**

- Data flows in one direction (parent → child), making the app predictable and easier to debug.

- **JSX (JavaScript XML)**

- A syntax extension that allows writing HTML-like code inside JavaScript.

Key Features

The diffing algorithm is React's smart way of figuring out **what changed** in the UI and **updating only that part** in the real DOM.

1. Virtual DOM

- React uses a Virtual DOM to optimize UI rendering. Instead of updating the entire real DOM directly, React:
- Creates a lightweight copy of the DOM (Virtual DOM).
- Compares it with the previous version to detect changes (diffing).
- Updates only the changed parts in the actual DOM (reconciliation), improving performance.

2. Component-Based Architecture

- React follows a component-based approach, where the UI is broken down into reusable components. These components:
- Can be functional or class-based.
- It allows code reusability, maintainability, and scalability.
- without requiring full-page reloads.

Key Features

3. JSX (JavaScript XML)

- React uses JSX, a syntax extension that allows developers to write HTML inside JavaScript. JSX makes the code:
 - More readable and expressive.
 - Easier to understand and debug.

4. One-Way Data Binding

- React uses one-way data binding, meaning data flows in a single direction from parent components to child components via props.
- This provides better control over data and helps maintain predictable behavior.

5. State Management

- React manages component state efficiently using the useState hook (for functional components) or this.state (for class components).
- State allows dynamic updates without reloading the page.

Key Features

6. React Hooks

- Hooks allow functional components to use state and lifecycle features without needing class components. Common hooks include:
- `useState`: for managing local state.
- `useEffect`: for handling side effects like API calls.
- `useContext`: for global state management.

7. React Router

- React provides React Router for managing navigation in single-page applications (SPAs).
- It enables dynamic routing

Key Features

Advantages

- High performance due to Virtual DOM.
- Reusable components reduce development time.
- Large community and ecosystem.
- Easy integration with other libraries or frameworks.

Key Features

Basic Example

```
import React from "react";  
import ReactDOM from "react-dom/client";
```

```
// Functional Component  
function Welcome(props) {  
  return <h1>Hello, {props.name}!</h1>;  
}
```

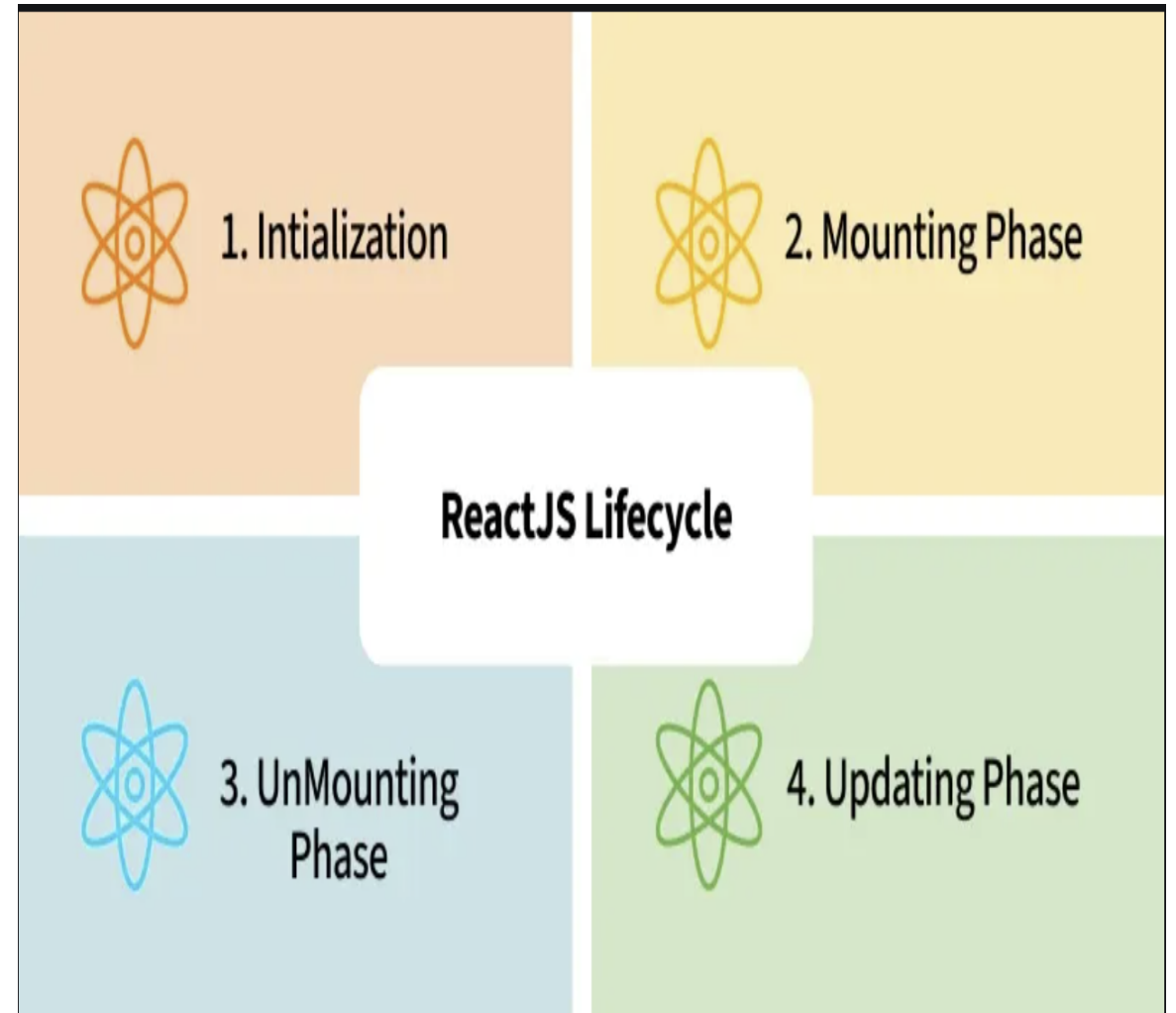
```
// Rendering the component  
const root = ReactDOM.createRoot(document.getElementById("root"));  
root.render(<Welcome name="React Developer" />);
```

How it works:

- Welcome is a **functional component** that takes props and returns JSX.
- ReactDOM.createRoot renders the component into the HTML element with id="root".

ReactJS Lifecycle

- Every React Component has a lifecycle of its own, the lifecycle of a component can be defined as the series of methods that are invoked in different stages of the component's existence.
- React automatically calls these methods at different points in a component's life cycle.
- Understanding these phases helps manage the state, perform side effects, and optimize components effectively.



ReactJS Lifecycle

1. Initialization

- This is the stage where the component is constructed with the given Props and default state.
- This is done in the constructor of a Component Class.

2. Mounting Phase

- Constructor: The constructor method initializes the component. It's where you set up the initial state and bind event handlers.
- render(): This method returns the JSX representation of the component. It's called during initial rendering and subsequent updates.
- componentDidMount(): After the component is inserted into the DOM, this method is invoked. Use it for side effects like data fetching or setting timers.

ReactJS Lifecycle

3. Updating Phase

- `componentDidUpdate(prevProps, prevState)`: Called after the component updates due to new props or state changes. Handle side effects here.
- `shouldComponentUpdate(nextProps, nextState)`: Determines if the component should re-render. Optimize performance by customizing this method.
- `render()`: Again, the `render()` method reflects changes in state or props during updates.

4. Unmounting Phase

- `componentWillUnmount()`: Invoked just before the component is removed from the DOM.
- Clean up resources (e.g., event listeners, timers).

Applications of React

- **Web Development:** React is used to build dynamic and responsive web applications, including social media platforms, e-commerce sites, and blogs.
- **Mobile Apps:** React Native allows developers to build mobile apps for iOS and Android using the same codebase.
- **Enterprise Applications:** React is used in building large-scale enterprise applications that require a highly interactive UI.
- **Dashboards and Data Visualizations:** React is great for building real-time dashboards and data visualization tools due to its high performance.

JSX Syntax Basics

JSX (**JavaScript XML**) lets you write HTML-like code inside JavaScript. It is **not HTML** — it's syntactic sugar for `React.createElement()`.

Rules:

- Must have **one parent element**.
- Use **camelCase** for attributes (className instead of class, onClick instead of onclick).
- JavaScript expressions go inside `{ }`.
- Self-close tags without children (`<img />`).
- Use `{ }` for dynamic values.

Example:

```
import React from "react";
import ReactDOM from "react-dom/client";

function App() {
  const name = "Alice";
  const items = ["Apple", "Banana", "Cherry"];

  return (
    <div>
      <h1>Hello, {name}!</h1>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li> /* Dynamic list
rendering */
        ))}
      </ul>
    </div>
  );
}

const root =
ReactDOM.createRoot(document.getElementById("root"
));
root.render(<App />);
```

React Components

React apps are built from **components** — reusable UI pieces.

Types of Components

- **Functional Components** (modern, preferred)
- **Class Components** (older, still supported)

Functional Component Example

```
function Greeting(props) {  
  return <h2>Hello, {props.name}!</h2>;  
}  
  
function App() {  
  return (  
    <div>  
      <Greeting name="Alice" />  
      <Greeting name="Bob" />  
    </div>  
  );  
}
```

Arguments can be passed into a component as **props**, which stands for properties.

Class Component Example

```
import React, { Component } from "react";  
  
class Greeting extends Component {  
  render() {  
    return <h2>Hello, {this.props.name}!</h2>;  
  }  
}  
  
class App extends Component {  
  render() {  
    return (  
      <div>  
        <Greeting name="Alice" />  
        <Greeting name="Bob" />  
      </div>  
    );  
  }  
}
```


State Management

State management is one of the important and unavoidable features of any dynamic application. React provides a simple and flexible API to support state management in a React component.

What is state?

- *State* represents the value of a dynamic properties of a React component at a given instance.
- React provides a dynamic data store for each component.
- The internal data represents the state of a React component and can be accessed using `this.state` member variable of the component.
- Whenever the state of the component is changed, the component will re-render itself by calling the *render()* method along with the new state.

React Hooks

- React Hooks are special functions provided by React to handle a specific functionality inside a React functional component.
- React provides a Hook function for every supported feature.
- For example, React provides `useState()` function to manage state in a functional component.
- When a React functional component uses React Hooks, React Hooks attach itself into the component and provides additional functionality.

The general signature of *useState()* Hook is as follows –

const [<state variable>, <state update function>] = useState(<initial value>);

For example, state management in clock component using Hooks can be done as specified below –

***const [currentDateTime, setCurrentDateTime] = useState(new Date());
setInterval(() => setCurrentDateTime(new Date()), 1000);***

React Hooks

- React Hooks are special functions provided by React to handle a specific functionality inside a React functional component.
- React provides a Hook function for every supported feature.
- For example, React provides `useState()` function to manage state in a functional component.
- When a React functional component uses React Hooks, React Hooks attach itself into the component and provides additional functionality.

The general signature of *useState()* Hook is as follows –

const [<state variable>, <state update function>] = useState(<initial value>);

For example, state management in clock component using Hooks can be done as specified below –

***const [currentDateTime, setCurrentDateTime] = useState(new Date());
setInterval(() => setCurrentDateTime(new Date()), 1000);***

React Routes for Navigation

- **React Routes are specifically used for navigation between pages or views** in a React application, especially in **Single Page Applications (SPAs)**.
- Instead of reloading the entire page, React Router handles navigation **client-side**, enabling smooth transitions and better performance.
- React itself doesn't have built-in routing, so developers use **react-router-DOM** to define routes, manage URLs, and render components based on the current path.

React Routes for Navigation- Example

// Install React Router

// npm install react-router-dom

import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

*function Home() {
 return <h1>Home Page</h1>;
}*

*function About() {
 return <h1>About Page</h1>;
}*

*function Contact() {
 return <h1>Contact Page</h1>;
}*

```
export default function App() {  
  return (  
    <BrowserRouter>  
      {/* Navigation Links */  
      <nav>  
        <Link to="/">Home</Link> |{" "  
        <Link to="/about">About</Link> |{" "  
        <Link to="/contact">Contact</Link>  
      </nav>  
  
      {/* Route Definitions */  
      <Routes>  
        <Route path="/" element={<Home />} />  
        <Route path="/about" element={<About />} />  
        <Route path="/contact" element={<Contact />} />  
      </Routes>  
    </BrowserRouter>  
  );  
}
```

In this example:

- **BrowserRouter** wraps the app to enable routing.
- **Link** changes the URL without reloading the page.
- **Routes** contains multiple Route definitions mapping paths to components.

React-Router Hooks

Routing

- Routing is the process of defining and managing navigation in an application so users can move between different views, pages, or sections within the app.
- Displays different components based on the URL.
- Enables seamless navigation without full page reloads.

React-Router Hooks

Types of Routing Hooks

These are 4 React Router Hooks in v5 that you can use in your React applications:

1. **useNavigate Hook**

- The [useNavigate](#) hook replaces the useHistory hook from React Router v5 for programmatic navigation. It provides a navigate function, which lets you navigate to different routes in your application.

2. **useParams Hook**

- The [useParams](#) hook is used to access the URL parameters from dynamic routes. It returns an object with key-value pairs representing the parameters.

React-Router Hooks

3. `useLocation` Hook

- The `useLocation` hook is provided by React Router and is used to access information about the current URL in your app. It is especially useful when you want your component's behavior or content to change depending on the current route, query parameters, or URL hash.

4. `useMatch` Hook

- The `useMatch` hook is a part of React Router v6 and is used to match the current URL to a given pattern. It provides a simpler and more flexible way to match routes compared to previous versions.

Companies using React

1. Facebook

Use Case: Core web interface, dynamic news feed, and interactive UI components.

Why React: React was originally developed by Facebook to handle complex UI updates efficiently.

2. Instagram

Use Case: Entire web app, including stories, explore page, and real-time updates.

Why React: Smooth rendering of media-heavy content and real-time interactions.

3. WhatsApp Web

Use Case: Web-based messaging interface.

Why React: Fast updates for chat messages and notifications without full page reloads.

4. Netflix

Use Case: Frontend for TV and web apps.

Why React: Optimized performance for low-bandwidth devices and fast rendering.

5. Airbnb

Use Case: Booking platform with interactive maps, filters, and listings.

Why React: Component reusability and dynamic search/filtering.

6. Uber

Use Case: Web dashboard for drivers, riders, and internal tools.

Why React: Real-time location updates and smooth UI transitions.

7. Discord

Use Case: Web and desktop chat application.

Why React: Handles frequent UI updates for messages, voice channels, and notifications.

8. Shopify

Use Case: E-commerce store builder and admin dashboard.

Why React: Modular UI components for store customization.

9. Pinterest

Use Case: Image-heavy social platform.

Why React: Efficient rendering of infinite scrolling and media grids.

10. Reddit

Use Case: Interactive feed, comments, and voting system.

Why React: Handles large-scale user interactions with minimal lag.